JORDAN UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF COMPUTER AND INFORMATION TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

CPE 591: Graduation Project I Report

BLUE-HANDED

Detection and Localization of Unauthorized Wireless Devices

Prepared by:

Yanal Al-Qudah (160714)

Rashida Khraise (161948)

Yazan Nayfeh (162303)

Jawad Bani Amer (163172)

Supervised

Dr. Husam Al-Qablan

Chapter 1 – Abstract :

There are existing solutions to the issue of cheating in exams by using Bluetooth devices like cameras, microphones and earphones. One of the most used solutions is as basic as the human observer to CCTV surveillance to the more advanced techniques like signal jammers or Lockdown browsers along with webcams to monitor face expressions and movement.

All of these solutions have flaws and many drawbacks, starting with the solution that depends solely on human observation of the students (Teacher or Proctors), this solution is the most traditional and widely used among all different educational institutes, the issue is that the proctor cannot detect or see the miniature modern hidden Bluetooth devices that the cheating students use nowadays. Moreover, human error is always prevalent in such processes. CCTV systems are even weaker when it comes to detecting cheating because the quality of the video stream isn't always high and a human has to actively watch the CCTV footage. Signal Jammers face a lot of legal challenges as it's generally illegal to straight on jam all the signals in the exam room. Lastly, software applications like browser lockdown along with webcams might offer a good solution but it is one that is suited to online digital exams, as traditional paper-based exams don't require the student to use a computer. Additionally, the cost might be expensive to have a dedicated webcam on each computer device in every exam hall.

Our proposed solution 'Blue-Handed' is an anti-cheat system that actively scans and monitors Bluetooth signals in the exam hall by using multiple (ESP32) microcontrollers that are spread across the room in a way that it can detect cheating students and even give the teacher or the administrator an approximate location of the cheating student detected, it works by continuously scanning the environment for Bluetooth signals emitted by cheating devices. With the help of the RSSI reading of the devices it estimates the distance to the device and indicates how close the cheater is and notifies the instructor via the web interface.

Our solution has many advantages and benefits over all the existing solutions beginning with the cost. It is relatively easy and inexpensive to set up multiple ESP32s. Furthermore, it is completely legal to scan and listen to the Bluetooth signals and packets transmitted in the air. It can additionally detect the smallest and unseen small devices like wireless earphones that a normal proctor wouldn't notice, also the system is automated and free of human errors as it relies on transmission signals. This system also works and is effective in paper-based exams as well.

Chapter 2 - Introduction :

Academic integrity is an elementary principle that is part of every academic institution's book of rules. However, several students have the propensity to violate this principle and engage in acts of academic misconduct. Cheating on an exam is one of the examples of academic dishonesty that happens commonly in universities all over the world, according to the International Center for Academic Integrity (ICAI, 2020) [1] more than 60 percent of university students freely admit to cheating in some form. This universal challenge creates an imbalance in how fairly students should be assessed. Moreover, it lowers the morale of those who work hard for their grades, while taking opportunities from their peers in the long run. This reoccurring phenomenon motivated us to work on a system that can be used to ensure exams can be conducted in an ethical and honest environment, preventing any distrust between students and faculty members.

To prevent academic misconduct from occurring in examinations. We have contributed to building a system that can be broken into:

- **A Distributed Hardware Sniffing Network**; a localized network of synchronized ESP32 microcontroller nodes acting as Bluetooth Low Energy (BLE) peripheral sniffers. This subsystem successfully scans and extracts critical signal metadata, such as MAC addresses and Received Signal Strength Indicators (RSSI), from the targeted cheating devices in real-time.

- **A Centralized Data Coordinator and Localization Engine**; it consists of the implementation of a Message Queuing Telemetry Transport (MQTT) based communication architecture on a central Raspberry Pi microprocessor to process the ESP32s' multi-node telemetry data. By utilizing the Log-Distance Path Loss model and Multilateration math, and integrating digital noise filtering, like Kalman filtering, to translate raw RSSI signals into accurate (x, y) coordinates.

- **Instantaneous Web-Based Visual Dashboard**; it includes an end-to-end web interface using Python micro-frameworks and WebSocket communication protocols. This provides invigilators with a dynamic graphical map of the examination environment, mapping detected cheating devices directly onto a visual representation of the hall layout.

The main goal of our solution is to be able to detect the presence of cheating devices that are commonly used in examination halls. Such as earpiece BLE devices. Ultimately preventing the widespread act of cheating across universities. The objectives that will help us to reach that goal are as follows:

- **Multi-Node Signal Scanning Deployment** To design and deploy a distributed physical array of ESP32 edge nodes capable of concurrent, real-time BLE signal scanning and metadata extraction.

- **Environment Calibration** To characterize the specific Radio Frequency (RF) propagation environment of the examination hall. This involves experimentally determining the path loss exponent and reference signal strength to calibrate the Log-Distance Path Loss model against signal attenuation caused by walls, furniture, and human presence.

- **Centralized Localization and Noise Filtering** To develop a centralized processing engine that combines our multi-node telemetry, implements digital filtering to bypass RSSI instability, and executes trilateration (or Multilateration) algorithms to compute precise coordinates.

- **Low-Latency Visual Monitoring (UI/UX)** To construct a real-time web-based dashboard that dynamically maps computed target locations onto a graphical representation of the room layout for invigilators.

Chapter 3 – Related Works :

In the following section, we present the related work that has previously been published in literature by researchers across various disciplines. Due to our project blending different fields of engineering; this section follows a thematic structure. With each part exhibiting a comparison analysis of the different technologies that exist and what we choose to go along with for our project.

The nature of our work follows that of an Indoor Positioning System (IPS). IPS was mainly developed to combat the technical limitations and failure of the Global Positioning System (GPS) in tracking items in indoor environments due to deep shadowing effects (Puricer and Kovar., 2007) [2]. Moreover, its diverse set of applications make it difficult to acquire a single unified architecture that is suitable for its dynamic and changing indoor environment (Pascacio et al., 2021) [3]. Several key aspects need to be accounted for when designing an IPS solution for a specific environment, all of which determine the required localization technology for the said system.

As Pascacio et al. demonstrated in their paper, there are several main requirements that determine the nature of the IPS' used technology. The two most prominent of these are:

- The accuracy and coverage that the system's environment requires
- The type of signal being used to track the targets

These two requirements go hand in hand. The type of signal chosen is the base of an IPS technology, it determines the range of space the system can cover and how close the measurements are to the actual positions. These signals have many classifications, but we will be focusing on Radio Frequency (RF) technology. Mainly Wireless Fidelity (WIFI) and BLE, relating to the fact that most cheating devices operate on these communication technologies.

According to a study that was conducted by Khoshrangbaf et al., (2025) [4] localization based on WIFI enabled systems have notably lower accuracy compared to BLE, with BLE tracking achieving at least a 44.12% increase in accuracy. Moreover, WIFI systems suffer from high error margins reaching over 250 cm for static targets and 600 cm for moving targets. Furthermore, technologies like BLE 5 have a coverage range up to 400 meters indoors, which fits the range that a university's examination halls require (Farahsari et al., 2022) [5].

After recognizing the type of signal an IPS tracks, the next step is determining the specific technique used to extract information from the RF signal to calculate the target's position. There exist many summarizations of IPS techniques in literature, one of them being the survey done by Zafari et al., (2019) [6] which discusses 8 different techniques famously used for localization. We will be focusing on the two widely used techniques which are based on extracting the RSSI from the signal. Received Signal Strength (RSS) – not to be confused with RSSI - is the absolute, physical measurement of power received by an antenna, usually measured in decibel-milliwatts (dBm) and is utilized to measure the distance between the transmitter and the receiver. RSSI on the other hand, is a manufacturer-specific, relative index used to estimate that power. RSS based localization requires trilateration (or N-point lateration). The receiver's RSS is used to estimate the absolute distance between the target and at least three reference points, afterwards basic geometry and trigonometry principles are applied to calculate the target's device location relative to the reference points. Another RSSI based technique is Fingerprinting, it relies on the collection of various RSSI measurements (called the offline phase). Once the system is deployed, the measurements taken at real-time (online phase) are compared with the offline measurements to estimate the user's location (Simon Yiu et al.,2017) [7]. Many have integrated the use of Machine

Learning (ML) and Deep Learning (DL) into this technique, making it more precise and reliable. However, as it highly depends on the predefined map, any changes in the environment require a remapping process (Hazzazi et al., 2026)8].

Other localization metrics can be used such as Time of Flight (ToF) or Time of Arrival (ToA) (Sverre Holm, 2012) [9]. They exploit the signal propagation time to calculate the distance between the transmitter and receiver. However, they both require strict synchronization between transmitters and receivers, which is a mismatch for our completely uncooperative monitoring network.

Lastly but not least, we proceed with the methods and algorithms that are used for computing the final coordinates of the target. For a Lateration based technique, a linearization method can be applied to the set of quadratic circle equations. Afterwards a Weighted Least Squares or a Total Least Squares approach can be applied to find the target's coordinates (Ismail Güvenc and Chia-Chin Chong, 2009) [10]. Alternatively, to bypass the linearization method, we could use a Taylor Series Estimation (Iterative Gauss-Newton) which uses the first-order Taylor series expansion to approximate the non-linear equations, calculates the error, and iteratively "steps" closer to the true coordinate. It provides us with more accurate results but requires larger processing power (Zhong Gaozhi et al., 2023) [11].

While for a Fingerprinting based technique, according to Suining He and Gary Chan (2016) [12] they categorized the traditional algorithms into deterministic algorithms and probabilistic algorithms. The first calculates the geometric distance of the target and the reference nodes and compares it to the data stored in the offline set, these methods can be easily implemented based on k nearest neighbors (k-NN) having the huge advantage of a low computational complexity. While the latter operates by assuming the indoor radio signals act as random variables that fluctuate constantly due to multipath fading, noise, and human bodies. It uses statistical density functions to calculate the probability that a target device is standing at a specific coordinate given the current signal reading. The main disadvantage is the complication in training and obtaining large enough datasets (Jochen Seitz et al., 2010) [13].

Before we conclude this section, it is worth mentioning that the use of Kalman filtering (KF) is advent among systems that depend on RSSI measurements as their localization technique. Sometimes, we may obtain false RSSI measurements due to their noisy nature leading to wrong and very unlikely estimated positions. As was depicted in the study done by Vicente Cantón Paterna et al., (2017) [14] the use of KF improved the estimation error compared to the other approaches.

Chapter 4 – Background :

4.1 Overview of Bluetooth-Based Cheating in Examinations

The problem that our project is targeting is Bluetooth-Based Cheating. This phenomenon has emerged significantly after some Bluetooth Low Energy (BLE) powered mini earphones often paired with smart watches, which is something that is hard to notice using the traditional methods because they're easy to hide during inspection before the exam for example.

This issue created a big challenge for professors trying to fairly evaluate student performance .

Our idea (Blue-Handed) addresses this by deploying multiple ESP32 boards across the examination hall to detect active Bluetooth devices, and forward any detected signals to a central raspberry pi server, which processes the data and presents alerts on a web-based monitoring dashboard.

4.2 Bluetooth Technology and BLE Advertising

Let's start by defining Bluetooth itself. Bluetooth is a wireless standard defined by the Bluetooth Special Interest Group (SIG). It operates in a 2.4 GHz band, which is used in most fields. Bluetooth Low Energy (BLE), introduced with

Bluetooth 4.0, is the one most relevant to this project. It is used in smart watches, earphones, and IoT devices as it's known for its low-power consumption. BLE devices work by broadcasting advertisement packets on three dedicated radio channels (37, 38, and 39). These packets contain information such as the device name, Media Access Control (MAC) address, and service identifiers. We will take advantage of this broadcast by configuring our ESP32s to scan for and capture these packets without pairing or alerting the transmitting device.

One practical challenge is MAC address randomization: modern BLE devices periodically rotate their MAC addresses to prevent passive tracking. Blue-Handed handles this by relying on Received Signal Strength Indicator (RSSI)-based proximity estimation rather than device identity, which allows detection regardless of address randomization.

4.3 Detection Based on Received Signal Strength Indicator (RSSI)

The Received Signal Strength Indicator (RSSI) measures the power level of a received radio signal in decibels relative to one milliwatt (dBm). A stronger RSSI (closer to 0 dBm) indicates a nearby device, while a weaker value (e.g., −90 dBm) suggests greater distance. The system estimates device proximity using the Log-Distance Path Loss model:

$$\mathbf{RSSI = RSSI_0 - 10 \times n \times \log_{10}(d)}$$

Where $RSSI_0$ is the reference signal strength at 1 meter, n is the path-loss exponent (typically 2–4 indoors), and d is the estimated distance in meters. In indoor environments, walls, furniture, and human bodies introduce noise into raw RSSI readings, so the system applies calibrated thresholds to classify devices as near (suspicious) or distant. Typical ranges are: −40 to −60 dBm within 0–2 m, −60 to −75 dBm at 2–5 m, and below −75 dBm beyond 5 m.

4.4 Kalman Filtering for RSSI Smoothing

Raw RSSI readings fluctuate significantly due to multipath propagation, reflections, and interference from other 2.4 GHz sources such as Wi-Fi. Without correction, these fluctuations produce inaccurate distance estimates and false alerts. The Blue-Handed system applies the Kalman Filter to smooth RSSI measurements before processing. The filter is an optimal recursive estimator that combines a predictive model with incoming measurements to minimize estimation error. It assumes the signal follows a slowly varying process with Gaussian noise, which suits indoor Bluetooth environments well. The filter update equation is:

$$\mathbf{\hat{x}_k = A\hat{x}_{k-1} + K(z_k - H\hat{x}_{k-1})}$$

Where $\hat{x}_k$ is the estimated RSSI at step k, A is the state transition matrix (1 for a constant model), K is the Kalman Gain balancing prediction and measurement, z_k is the raw RSSI reading, and H is the observation matrix (1 for direct measurement). Applying this filter reduces signal fluctuations, stabilizes readings over time, improves distance estimation accuracy, and lowers the false positive rate. Zhou et al. (2017) [15] and Daniswara et al. (2024) [16] both confirmed that Kalman-filtered RSSI values yield significantly better positioning accuracy in multi-node BLE systems, particularly in dense device environments comparable to an examination hall.

4.5 Trilateration and Device Localization

With multiple ESP32 nodes placed at known positions in the examination hall, the system collects RSSI readings from the same device at three or more nodes simultaneously. Trilateration uses these readings to estimate the device's physical location: each node defines a circle centered on its position with a radius equal to the estimated distance, and the device's position is estimated at the intersection of these circles. Because RSSI noise prevents a perfect intersection, a weighted

least-squares method is applied, giving greater weight to nodes with stronger signal readings. The resulting position is displayed on the hall's floor plan in the dashboard, allowing supervisors to pinpoint the approximate location of a suspicious device.

4.6 Anomaly Detection and Suspicious Device Classification

Not every detected Bluetooth device represents a cheating attempt; invigilators, nearby offices, and passing individuals may also appear in scans. The system therefore applies the following criteria to classify a device as suspicious:

- RSSI threshold: the device is estimated to be within a defined proximity (e.g., within 3 meters of a node inside the hall)
- Device profile: the device advertises BLE audio characteristics consistent with earphones
- Persistence: the device appears consistently across multiple scan cycles, confirming physical presence rather than a brief pass-by
- Whitelist filtering: pre-registered devices belonging to invigilators are excluded from alerts

4.7 Real-Time Monitoring

The ESP32 nodes scan continuously throughout the examination, transmitting data to the Raspberry Pi via the Message Queuing Telemetry Transport (MQTT) protocol over the local Wi-Fi network. The server processes incoming readings, applies filtering and detection logic, and pushes updates to the dashboard in real time via WebSocket. Any newly activated Bluetooth device is detected and reported within seconds, enabling supervisors to act promptly.

Chapter 5 - Hardware and Software Components :

5.1 Hardware Components

5.1.1 Espressif Systems ESP32 Microcontroller

Multiple ESP32 modules are deployed as distributed BLE scanning nodes throughout the examination hall. Each unit independently performs passive BLE scanning and forwards collected data to the central server via Wi-Fi.

Technical Specifications:

Parameter	Specification
Processor	Xtensa dual-core 32-bit LX6, up to 240 MHz
On-Chip SRAM	520 KB
Flash Memory	4 MB (external SPI flash)

Parameter	Specification
Wi-Fi	IEEE 802.11 b/g/n, 2.4 GHz
Bluetooth	v4.2 Classic + BLE
Operating Voltage	3.3 V logic; 5 V via USB
Operating Temperature	−40°C to +85°C
Dimensions (WROOM-32)	18 × 25.5 × 3.1 mm

Role in the System:

- Passive BLE scanning to capture advertisement packets from nearby devices
- Recording RSSI values for each detected device
- Publishing scan data to the Raspberry Pi via MQTT over Wi-Fi
- Running C/C++ firmware built with the Arduino framework and ESP-IDF

Power Supply:

Each ESP32 node is powered by a portable rechargeable lithium-ion battery pack via USB-to-micro-USB, delivering a regulated 5 V that the module regulates internally to 3.3 V. This makes each node fully wireless and independently placeable throughout the hall.

5.1.2 Raspberry Pi 4 Model B

The Raspberry Pi 4 Model B (RPi 4B) acts as the central server of the Blue-Handed system. It receives data from all ESP32 nodes, runs signal processing and detection algorithms, manages the database, hosts the web dashboard.

Technical Specifications:

Parameter	Specification
Processor	Broadcom BCM2711, quad-core Cortex-A72 (ARM v8), 64-bit, 1.5 GHz
RAM	4 GB LPDDR4-3200 SDRAM (recommended for this project)
Wi-Fi	Dual-band 2.4 / 5.0 GHz IEEE 802.11 b/g/n/ac
Bluetooth	5.0 + BLE (built-in)

Parameter	Specification
Ethernet	Gigabit Ethernet
USB Ports	2 × USB 3.0 + 2 × USB 2.0
Storage	MicroSD card slot
GPIO	40-pin header
Operating System	Raspberry Pi OS (64-bit, Debian-based Linux)
Dimensions	85.6 × 56.5 mm

Role in the System:

- Receiving scan data from ESP32 nodes via the MQTT broker
- Applying Kalman filtering and trilateration
- Running anomaly detection to classify suspicious devices
- Storing records in the SQLite database
- Hosting the Flask web server and monitoring dashboard

Power Supply:

The RPi 4B requires 5 V / 3 A delivered via its USB Type-C (USB-C) port using an official Raspberry Pi power adapter connected to a wall outlet, ensuring stable continuous operation.

5.1.3 Communication Protocols

Message Queuing Telemetry Transport (MQTT):

MQTT is a lightweight publish-subscribe protocol designed for constrained IoT devices. ESP32 nodes act as publishers, sending scan results to a topic on the MQTT broker running on the Raspberry Pi. The server subscribes to that topic and processes messages as they arrive. MQTT's minimal overhead makes it significantly more efficient than HTTP polling for this type of continuous data stream.

Hypertext Transfer Protocol (HTTP) and WebSocket:

The Flask server uses HTTP for initial dashboard delivery and WebSocket (via Flask-SocketIO) for real-time bidirectional updates. WebSocket maintains a persistent connection between the server and the browser, enabling instant push of detection events to supervisors without page refreshes.

Transmission Control Protocol / Internet Protocol (TCP/IP):

All system communication — MQTT and WebSocket alike — runs over the Transmission Control Protocol/Internet Protocol (TCP/IP) stack on a dedicated local Wi-Fi network to which all ESP32 nodes and the Raspberry Pi are connected.

5.1.4 Hardware Components Summary

Component	Qty	Role	Power Source
ESP32 (WROOM-32)	3–6	BLE Passive Scanner Node	Rechargeable Battery Pack (USB)
Raspberry Pi 4 Model B (4 GB)	1	Central Server & Processing Unit	USB-C Adapter (5 V / 3 A)

5.2 Software Components

5.2.1 Programming Languages

C / C++:

ESP32 firmware is written in C/C++ using the Arduino framework within the ESP-IDF environment. This gives direct access to the hardware BLE stack and enables the passive scan loop, RSSI formatting, and MQTT publishing.

Python:

Python runs on the Raspberry Pi and handles the MQTT subscriber, Kalman filtering, anomaly detection, database management, and the Flask web server. It was chosen for its rich library ecosystem and strong IoT and data-processing community support.

5.2.2 Web-Based Monitoring Dashboard

The dashboard is a web application hosted on the Raspberry Pi, accessible from any browser on the same local network. It displays detected devices, RSSI values, estimated locations, and active alerts in real time.

Flask:

Flask is a lightweight Python web framework used as the dashboard backend. It handles routing, serves the dashboard page, and integrates with Flask-SocketIO for WebSocket support. Its minimal overhead is well suited to the resource constraints of the Raspberry Pi.

Flask-SocketIO:

Flask-SocketIO adds full-duplex, event-driven WebSocket communication to Flask. Whenever the server detects a new suspicious device, it emits a Socket.IO event that is instantly pushed to all connected dashboard clients without requiring a page reload.

Front-End Technologies:

- HyperText Markup Language (HTML): structures the device table, alert panel, and floor plan display
- Cascading Style Sheets (CSS): styles the interface and visually differentiates suspicious from normal entries
- JavaScript: handles client-side WebSocket listeners and dynamically updates the dashboard as new data arrives
- Chart.js: renders real-time RSSI signal strength graphs per detected device

5.2.3 Database

SQLite is used as the embedded database. Being serverless and file-based, it requires no additional database process and is ideal for a self-contained Raspberry Pi deployment. The database stores detected MAC addresses, raw and filtered RSSI readings with timestamps, estimated positions, alert records, and the invigilator device whitelist. The built-in Python sqlite3 module handles all database interactions, enabling post-examination audit of session activity.

5.2.4 Key Python Libraries

Library	Purpose
paho-mqtt	MQTT client; subscribes to the broker and receives ESP32 scan data
flask	Web framework; serves the dashboard and handles HTTP routing
flask-socketio	WebSocket extension; real-time data push to the browser dashboard
sqlite3	Built-in module; manages the SQLite database
numpy	Kalman filter matrix operations and trilateration calculations
RPi.GPIO	GPIO control
eventlet	Async networking; required by Flask-SocketIO for concurrent WebSocket handling

5.2.5 ESP32 Firmware Libraries

Library	Purpose
BLEDevice (Arduino-ESP32)	BLE scanner API for passive advertising packet capture

Library	Purpose
PubSubClient	MQTT client; publishes scan data from the ESP32 to the broker
WiFi (Arduino-ESP32)	Manages Wi-Fi connectivity for the ESP32
ArduinoJson	Serializes and parses JSON-formatted MQTT messages

5.2.6 Development Tools

Visual Studio Code (VS Code):

VS Code is the primary Integrated Development Environment (IDE) used across the project. The PlatformIO extension manages the ESP32 build toolchain, library dependencies, and device flashing, while the integrated terminal and debugger support Raspberry Pi development.

GitHub:

GitHub hosts the project repository for version control and collaboration, covering both the ESP32 firmware and the Raspberry Pi server application. It tracks all changes and maintains a full development history.

5.2.7 Operating System

The Raspberry Pi 4 runs Raspberry Pi OS (64-bit), a Debian-based Linux distribution maintained by the Raspberry Pi Foundation. It provides native support for all required Python libraries, the Mosquitto MQTT broker, and the Flask server, offering a stable platform for continuous operation.

5.2.8 Software Components Summary

Component	Platform	Purpose
C/C++ (Arduino / ESP-IDF)	ESP32	BLE scanning firmware
Python 3	Raspberry Pi	Server logic, signal processing, database management
Flask	Raspberry Pi	Web server and dashboard backend
Flask-SocketIO	Raspberry Pi	Real-time WebSocket communication
paho-mqtt	Raspberry Pi	MQTT subscriber
Mosquitto MQTT Broker	Raspberry Pi	MQTT message broker
SQLite3	Raspberry Pi	Persistent storage

Component	Platform	Purpose
NumPy	Raspberry Pi	Kalman filter and trilateration
HTML / CSS / JavaScript	Browser	Dashboard front-end
Chart.js	Browser	Real-time RSSI graphs
PubSubClient	ESP32	MQTT publishing
VS Code + PlatformIO	Development PC	IDE and ESP32 flashing
GitHub	Development PC	Version control

Chapter 6 – Proposed Work :

6.1 Use Case Diagram

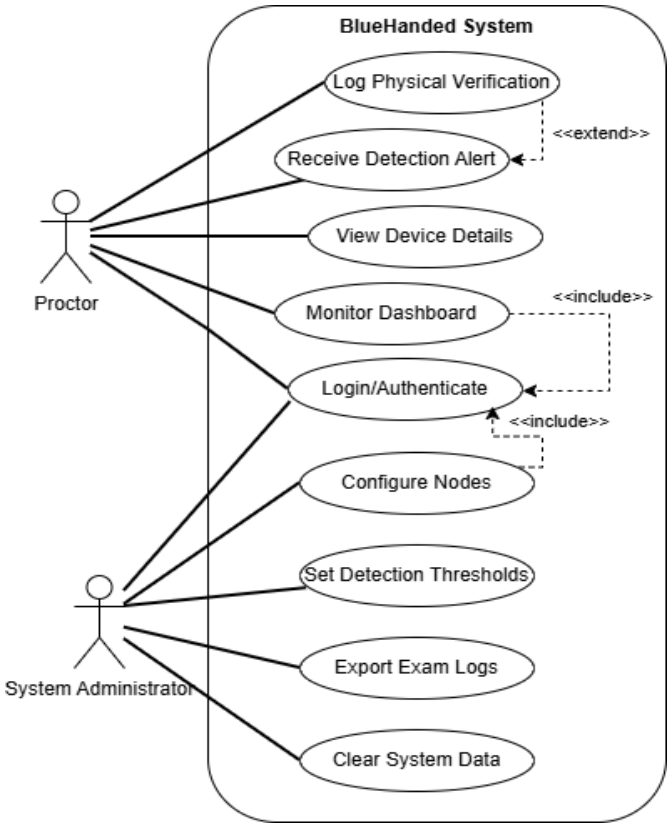


Figure (1): Use Case Diagram

6.1.1 Proctor Use Cases

- **Login / Authenticate**: A security check point. The monitor must validate his credentials before gaining access to the monitoring software to ensure that no unwanted persons are allowed into the detection system.

- **Monitor Dashboard**: This is the primary responsibility of the proctor while conducting the examination, where the user accesses the centralized graphical interface of the Raspberry Pi. This interface gives a real-time overview of the working of all active ESP32 nodes along with any Bluetooth traffic in the examination hall.

- **Receive Detection Alert**: System-initiated event. If the ESP32 node detects a Bluetooth transmission (that of an earphone, for instance), whose RSSI is above the pre-configured threshold, a system alert is triggered which generates a visual and audible alarm on the proctor's dashboard regarding the location of the anomaly.

- **View Device Details**: Following the trigger signal, the proctor jumps straight into the system where he checks details of the triggered device. He should be able to check on the MAC address, device name (if available), and the exact RSSI level to determine the distance of the device from the closest ESP32 sensor.

- **Log Physical Verification (Extends "Receive Detection Alert"):** Since it is necessary to confirm the results by humans to prevent cheating using spoofing, it becomes crucial for this process to happen. After confirming the alert manually, the proctor enters the result into the system, which can be marked as a “true positive” or a “false alarm.”

6.1.2 System Administrator Use Cases

- **Login / Authenticate:** Just like the proctor, the system administrator has to log in with elevated credentials to access the backend configuration and logging tools.

- **Configure Nodes:** It is an exam prerequisite task. The administrator will match up the distributed ESP32 microcontrollers to the Raspberry Pi network, which ensures that there is effective communication and everything is on before the students arrive.

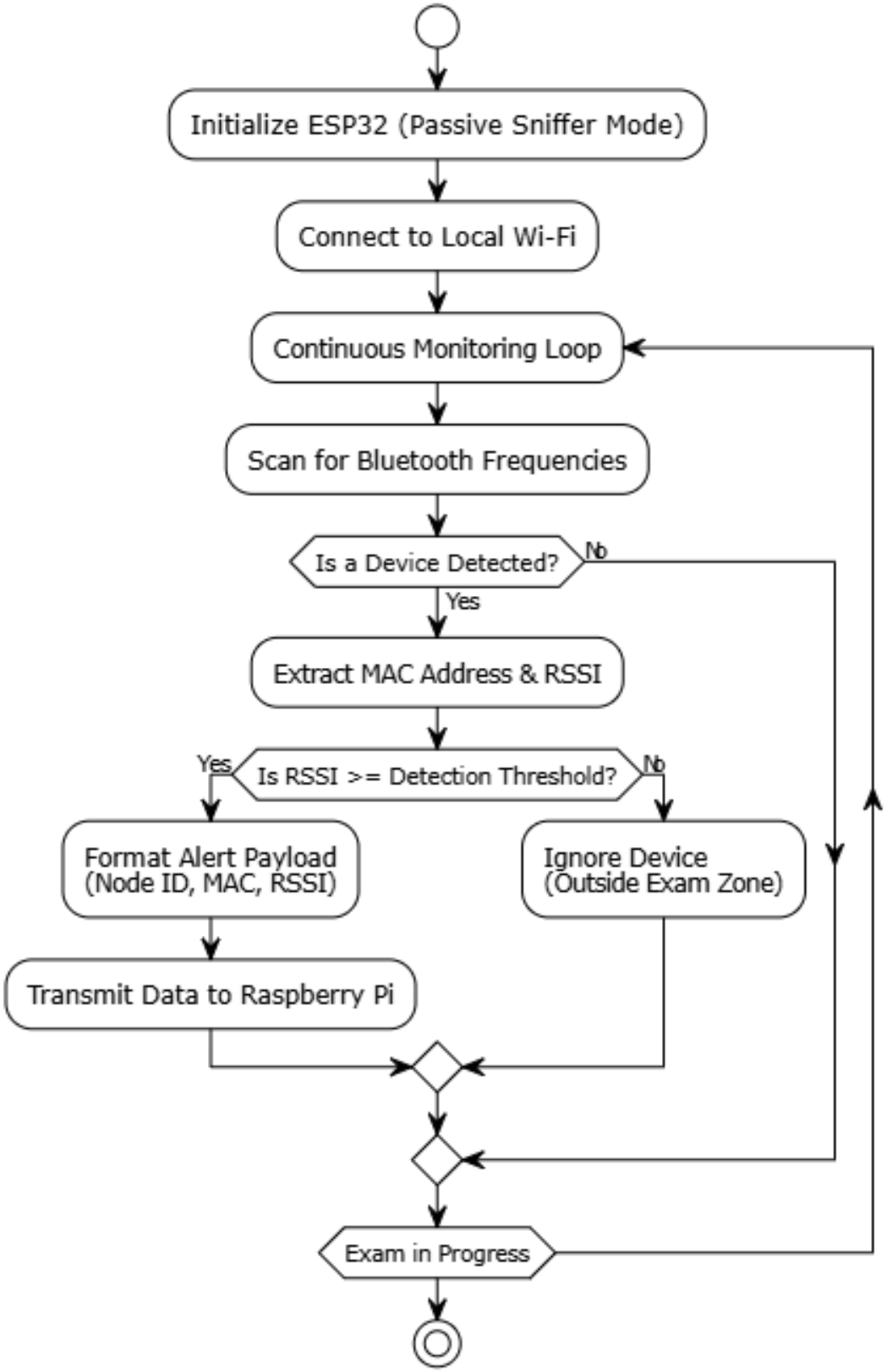
- **Set Detection Thresholds:** The administrator makes fine adjustments to the sensitivity settings in accordance with the size of the examination room. With appropriate RSSI thresholds set, the system detects concealed earphones while ignoring other devices that may be present outside in the hallway.

- **Export Exam Logs:** This is an assignment after the examination. The administrator collects and downloads the event history along with the MAC addresses, alert timestamps, and verification comments of the proctor into a report.

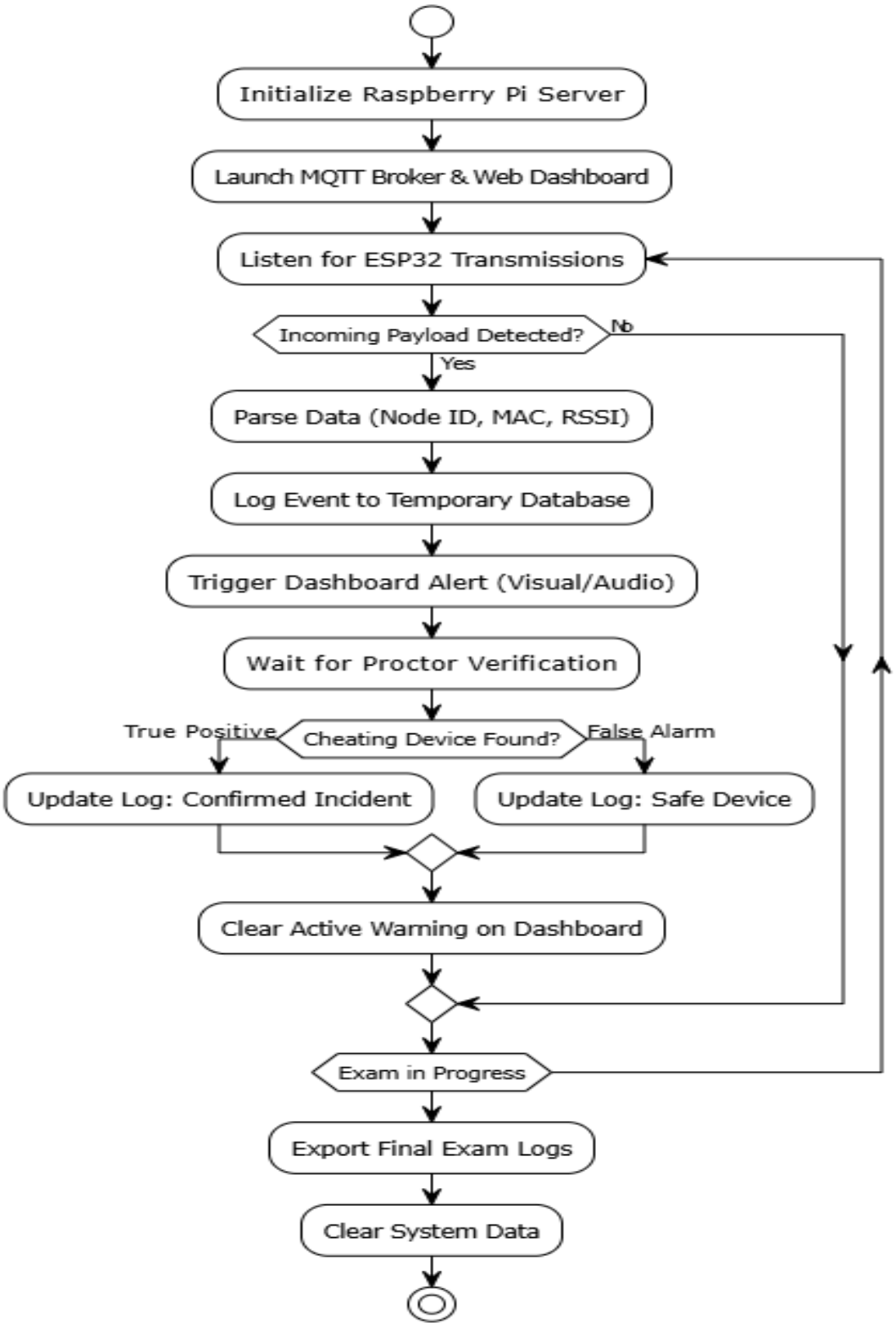
- **Clear System Data:** In order to maintain compliance with data protection laws and regulations, the temporary logs and MAC address cache will be deleted from the database of the Raspberry Pi after the review period of the examination ends.

6.2 Flow Chart:

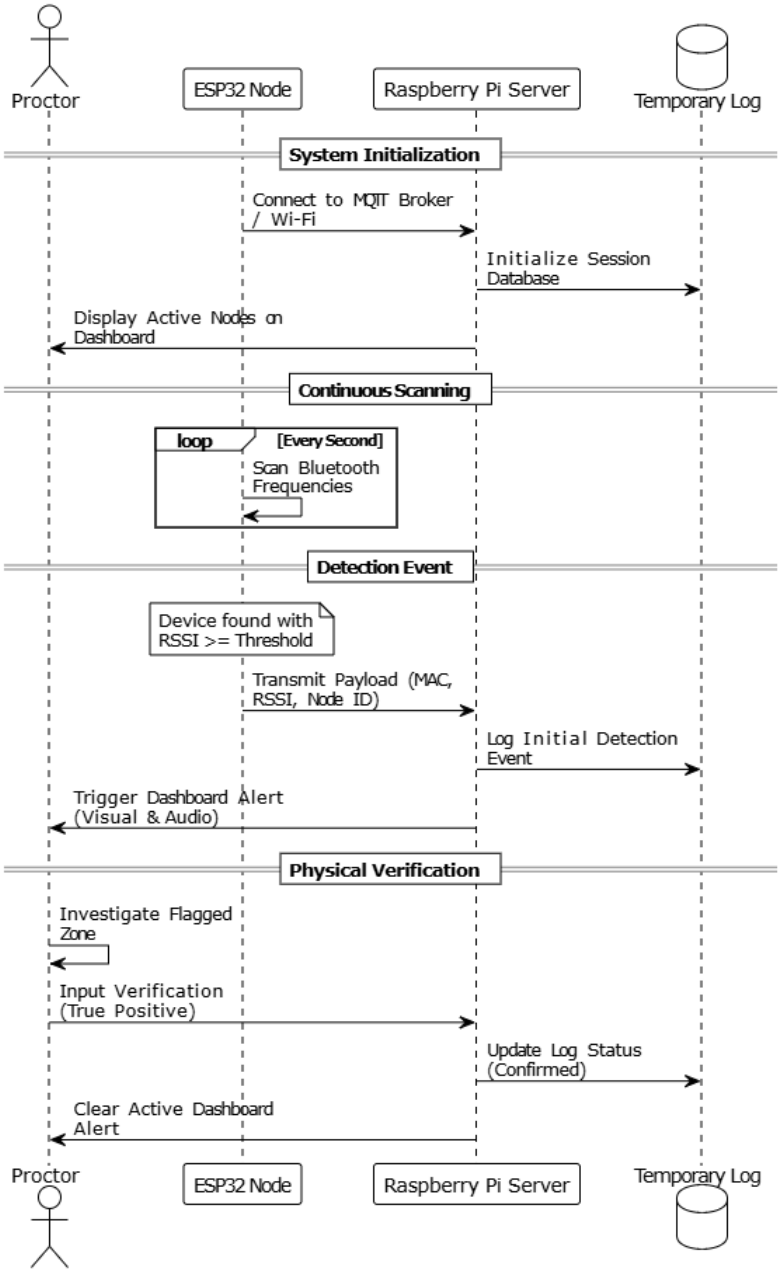
6.2.1 ESP32



6.2.1 Raspberry Pi



6.3 Sequence Diagrams



Chapter 7 – Professional Constraints :

7.1 Privacy Constraints

- Data Minimization: The ESP32s are programmed to operate solely as passive sniffers that capture information from the cheating device. The collected information consists of the following: MAC Addresses, Names (when available) and RSSI values.

- No Payload Interception: The system does not record the actual audio or personal data being transmitted through the earphones.
- Data Retention: The logged information by the Raspberry Pi can only remain saved throughout the test period and the following review stage. The minute that cheating has been dealt with, all the information will be erased.

7.2 Security Constraints

- Network Security: Communication from the ESP32 nodes to the Raspberry Pi needs to be secured. If this communication occurs on a local Wi-Fi network, encryption of the data packets is required (TLS/SSL). This would prevent students from intercepting the network traffic.
- Anti-Spoofing: It should be able to detect any form of interference, for instance, attempts to spoof the MAC addresses of students in order to create false alarms. However, this problem can be solved by operating under the assumption that automated alerts, based on RSSI limits, are just warnings and not a form of automated punishment. The final confirmation will always come from the proctor in physical form.
- Physical Security: These components must be placed at places that cannot be reached by the students inside the examination hall. Preventing any kind of deliberate attempt by the students to damage or unplug the nodes

7.3 Safety Constraints

- Radio Frequency Exposure Limits: The ESP32s are low-power devices. Since they do not transmit any high-power signal but merely receive information, they present no health risks whatsoever to the students and even invigilators sitting there.
- Operational Non-Interference: In contrast to the active jamming signals, the Blue-Handed system will not affect personal items that are protected and sensitive, such as a pacemaker worn by a student or his/her hearing aid.
- Electrical and Physical Safety: The hardware works using low DC voltage. Physical safety is ensured when all equipment is firmly fixed to avoid any fall, and all power lines are well connected.

7.4 Economy Constraints

- Hardware Cost Efficiency: Blue-Handed employs inexpensive and easily available microcontrollers (ESP32) and one main microcontroller (Raspberry Pi). Therefore, it is economical for learning institutions.
- Scalable Infrastructure: It ensures that universities have an option to expand their testing facilities without the need for complicated wiring processes or extensive modifications to their examination halls by ensuring the use of low power equipment.

7.5 Regulatory and legal Constraints

- Telecommunications Compliance: Blue-Handed only listens to public Bluetooth frequencies and avoids illegal electronic surveillance.

Chapter 8 – Deliverables :

To ensure systematic execution, hardware deployment, algorithmic calibration, and final verification of the Blue-Handed system during the Graduation Project II (GP2) phase, deliverables are organized chronologically from inception to final defence. The table below outlines the core execution milestones, target delivery dates, and detailed technical components associated with each phase:

Phase / Target Date	Deliverable Name	Description & Key Structural Components
Phase 1 July 2026	Hardware Procurement & Component Bench Testing	• Sourcing of physical hardware components. • Verify and test power consumption constraints with battery packs. • Configuring the 64-bit Raspberry Pi OS environment.
Phase 2 September 2026	Distributed Sniffing Network & MQTT Integration	• Deployment of the wireless localized network • Enabling passive BLE scanning for MAC and RSSI extraction. • Establishing JSON data publishing over Mosquitto MQTT.
Phase 3 November 2026	Centralized Localization Engine & Filter Deployment	• Deploying 1D Kalman filters for RSSI smoothing. • Calibrating the Log-Distance Path Loss environment model. • Executing NumPy-based multilateration for (x, y) coordinates.
Phase 4	Full-Duplex Web Dashboard &	• Building the Flask backend web server.

Phase / Target Date	Deliverable Name	Description & Key Structural Components
December 2026	Persistent Storage Suite	• Implementing Flask-SocketIO for real-time data pushes. • Designing SQLite databases for session logs and whitelists.
Phase 5 January 2027	System Calibration, Field Testing & Security Hardening	• Conducting full field tests simulating an examination hall. • Fine-tuning thresholds to counter MAC randomization or any other real-life challenges. • Implementing TLS/SSL encryption and physical node security.
Phase 6 February 2027	Final GP2 Comprehensive Report & Live Jury Demonstration	• Compiling the final technical report. • Preparing presentation slides and data validation logs. • Conducting a live end-to-end prototype defence for the jury.

Chapter 9 – Roles :

Yanal Al-Qudah:

Responsible for embedded systems programming, Setting up the ESP32 nodes, Bluetooth scanning implementation, extraction of data, and communication protocols.

Rashida Khraise

Responsible for backend development, setting up the Raspberry Pi server, creation and management of the database system, and smooth integration of the diverse systems.

Yazan Nayfeh

Responsible for developing the localization algorithms, RSSI processing, implementing the Kalman filtering, and heatmap generation.

Jawad Bani Amer

Responsible for developing the dashboard design(UI/UX), testing procedures, documentation, workflow management and project presentation preparation.

Chapter 10 – Bibliography:

1. International Center for Academic Integrity. (2020). *Facts & statistics*. https://www.academicintegrity.org/aws/ICAI/pt/sp/facts

2. Puricer, Pavel & Kovář, Pavel. (2007). Technical Limitations of GNSS Receivers in Indoor Positioning. 1 - 5. 10.1109/RADIOELEK.2007.371487.

3. Pascacio, P., Casteleyn, S., Torres-Sospedra, J., Lohan, E. S., & Nurmi, J. (2021). Collaborative Indoor Positioning Systems: A Systematic Review. *Sensors*, *21*(3), 1002. https://doi.org/10.3390/s21031002

4. Khoshrangbaf, M., Akram, V. K., Challenger, M., & Dagdeviren, O. (2025). An Experimental Evaluation of Indoor Localization in Autonomous Mobile Robots. *Sensors*, *25*(7), 2209. https://doi.org/10.3390/s25072209

5. Farahsari, P. S., Farahzadi, A., Rezazadeh, J., & Bagheri, A. (2022). A Survey on Indoor Positioning Systems for IoT-Based Applications. IEEE Internet of Things Journal, 9(10), 7680–7699. https://doi.org/10.1109/jiot.2022.3149048

6. Zafari, F., Gkelias, A., & Leung, K. K. (2019). A Survey of Indoor Localization Systems and Technologies. IEEE Communications Surveys & Tutorials, 21(3), 2568–2599. https://doi.org/10.1109/comst.2019.2911558

7. Simon Yiu, Marzieh Dashti, Holger Claussen, Fernando Perez-Cruz (2017), Wireless RSSI fingerprinting localization, Signal Processing, Volume 131, Pages 235-244, ISSN 0165-1684, https://doi.org/10.1016/j.sigpro.2016.07.005.

8. Hazzazi, M.M., Shukla, P.K., Shukla, P.K. *et al.* A secure and high-accuracy indoor positioning system using deep temporal fusion transformer and Galois field encryption. *J Big Data* **13**, 35 (2026). https://doi.org/10.1186/s40537-025-01356-7

9. S. Holm, "Ultrasound positioning based on time-of-flight and signal strength," 2012 International Conference on Indoor Positioning and Indoor Navigation (IPIN), Sydney, NSW, Australia, 2012, pp. 1-6, doi: 10.1109/IPIN.2012.6418728

10. Guvenc, I., & Chong, C.-C. (2009). A Survey on TOA Based Wireless Localization and NLOS Mitigation Techniques. IEEE Communications Surveys & Tutorials, 11(3), 107–124. https://doi.org/10.1109/surv.2009.090308

11. Gaozhi, Z., Sa, X., Jie, Z., Xiangyu, W., Changbo, H. (2022). A Direct Location Algorithm Based on Gauss Newton Iteration. In: Chenggang, Y., Honggang, W., Yun, L. (eds) Mobile Multimedia Communications. MobiMedia 2022. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, vol 451. Springer, Cham. https://doi.org/10.1007/978-3-031-23902-1_4

12. He, S., & Chan, S.-H. G. (2016). Wi-Fi Fingerprint-Based Indoor Positioning: Recent Advances and Comparisons. IEEE Communications Surveys & Tutorials, 18(1), 466–490. https://doi.org/10.1109/comst.2015.2464084

13. Seitz, J., Vaupel, T., Jahn, J., Meyer, S., Boronat, J.G., & Thielecke, J. (2010). A Hidden Markov Model for urban navigation based on fingerprinting and pedestrian dead reckoning. *2010 13th International Conference on Information Fusion*, 1-8.

14. Cantón Paterna, V., Calveras Augé, A., Paradells Aspas, J., & Pérez Bullones, M. (2017). A Bluetooth Low Energy Indoor Positioning System with Channel Diversity, Weighted Trilateration and Kalman Filtering. Sensors, 17(12), 2927. https://doi.org/10.3390/s17122927

15. Zhou, C., Yuan, J., Liu, H., & Qiu, J. (2017). Bluetooth Indoor Positioning Based on RSSI and Kalman Filter. Wireless Personal Communications, 96, 4115 - 4130.

16. Musayyanah, Musayyanah & Daniswara, Hendra & Susanto, Pauladie & Harianto, Harianto. (2024). The Influence of Kalman Filtering on the Received Signal Strength Indicator in Multi-node Bluetooth Low Energy Communications. Emitor: Jurnal Teknik Elektro. 108-114. 10.23917/emitor.v24i2.2355.